

Creating an Amazon ElastiCache Cache Using Amazon RDS DB Instance Settings

To Begin with the Lab

Summary of the Lab

In this lab, we created a custom AWS environment using a VPC and launched an EC2 instance to act as the application server. We deployed an Amazon RDS MySQL database and loaded around 210,000 sample records into it. Next, we provisioned an Amazon ElastiCache for Redis cluster and established secure connectivity between EC2, RDS, and Redis using security groups. Using Python scripts, we migrated data from RDS to Redis and verified that the data was successfully cached. Finally, we queried data from both RDS and Redis through EC2 and compared their response times to understand the role of Redis as a caching layer for improving application performance and reducing database load.

- Go to AWS
- Navigate to VPC
- Click Create VPC
- In VPC Settings
- Choose VPC and more
- Name the VPC

- Number of Availability Zones
 - 2
- Number of Public Subnets
 - 2
- Number of Private Subnets
 - 2

Number of Availability Zones (AZs) [Info](#)

Choose the number of AZs in which to provision subnets. We recommend at least two AZs for high availability.

1 | **2** | 3

[Customize AZs](#)

Number of public subnets [Info](#)

The number of public subnets to add to your VPC. Use public subnets for web applications that need to be publicly accessible over the internet.

0 | **2**

Number of private subnets [Info](#)

The number of private subnets to add to your VPC. Use private subnets to secure backend resources that don't need public access.

0 | **2** | 4

VPC [Show details](#)

Your AWS virtual network

RDS-VPC-vpc

Subnets (4)

Subnets within th

us-east-1a

A RDS-VPC-s

A RDS-VPC-s

us-east-1b

B RDS-VPC-s

B RDS-VPC-s

- NAT Gateway Type: Zonal
- NAT Gateways: In 1 AZ
- VPC Endpoints
 - None

NAT gateways (\$) - *updated* [Info](#)

NAT gateway allows private resources to access the internet from any availability zone within a VPC, providing a single managed internet exit point for the entire region. Additional charges apply.

None | Regional - new | **Zonal**

Introducing regional NAT gateway

AWS now offers a multi-AZ NAT Gateway, eliminating the need for separate NAT Gateways across availability zones.

NAT gateways (\$) [Info](#)

Choose the number of Availability Zones (AZs) in which to create NAT gateways. Note that there is a charge for each NAT gateway

In 1 AZ | 1 per AZ

VPC endpoints [Info](#)

Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.

None | S3 Gateway

Preview

VPC [Show details](#)

Your AWS virtual network

project-vpc

Subnets (4)

Subnets within this VPC

us-east-1a

A project-subnet-pub

A project-subnet-priv

us-east-1b

B project-subnet-pub

B project-subnet-priv

- Click Create VPC
- Now Create three Security Group
- One for the EC2 and other two for the RDS and Redis
- Choose the VPC that we have created (RDS-VPC)
- Inbound rule for EC2-SG
- If My IP doesn't work then choose Anywhere IPv4

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

EC2-SG

Name cannot be edited after creation.

Description [Info](#)

EC2 Access

VPC [Info](#)

vpc-0e0462813e3d4bdb0 (RDS-VPC-vpc)

Inbound rules [Info](#)

Type	Protocol	Port range	Source	Description - optional
SSH	TCP	22	My IP	

[Delete](#)

- Inbound rule for RDS-SG

Basic details

Security group name [Info](#)
 RDS-SG
Name cannot be edited after creation.

Description [Info](#)
 RDS Access

VPC [Info](#)
 vpc-0e0462813e3d4bdb0 (RDS-VPC-vpc)

Inbound rules [Info](#)

Type	Protocol	Port range	Source	Action	Option - optional
MYSQL/Aurora	TCP	3306	Custom	Allow	

Source dropdown menu:

- Use: "sg-05fe39762e6ef8b3d"
- CIDR blocks
- Security Groups
 - EC2-SG | sg-05fe39762e6ef8b3d
- Prefix lists

Search bar: Q sg-05fe39762e6ef8b3 X

Selected items: sg-05fe39762e6ef8b3d X

[Delete](#)

- Inbound Rule for Redis-SG

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)
 Redis-SG
Name cannot be edited after creation.

Description [Info](#)
 Redis Access

VPC [Info](#)
 vpc-0e0462813e3d4bdb0 (RDS-VPC-vpc)

Inbound rules [Info](#)

Type	Protocol	Port range	Source	Action	Option - optional
Custom TCP	TCP	6379	Custom	Allow	

[Add rule](#)

Source dropdown menu:

- Use: "sg-05fe39762e6ef8b3d"
- CIDR blocks
- Security Groups
 - EC2-SG | sg-05fe39762e6ef8b3d
 - EC2-SG
- Prefix lists

Search bar: Q sg-05fe39762e6ef8b3 X

Selected items: sg-05fe39762e6ef8b3d X

[Delete](#)

- Now Let's create an EC2
- AMI: Amazon Linux
- Instance Type: t3.micro
- Either create a new Key Pair or use the existed one
- Choose the VPC that you have created (RDS VPC)
- Subnet: RDS-VPC-subnet-public1-us-east-1a
- Note: Choose any public subnet.
- Auto Assign Public IP: Enable

The screenshot shows the 'Network settings' and 'Summary' tabs for an EC2 instance configuration in the AWS Management Console.

Network settings:

- VPC:** vpc-0e0462813e3d4bdb0 (RDS-VPC-vpc)
- Subnet:** subnet-0ec4143f8cf663964 (RDS-VPC-subnet-public1-us-east-1a)
- Auto-assign public IP:** Enable
- Firewall (security groups):** Select existing security group (EC2-SG)
- Common security groups:** EC2-SG sg-05fe39762e6ef8b3d

Summary:

- Number of instances:** 1
- Software Image (AMI):** Amazon Linux 2023 AMI 2023.12....read more
- Virtual server type (instance type):** t3.micro
- Firewall (security group):** EC2-SG
- Storage (volumes):** 1 volume(s) - 8 GiB

Buttons: Cancel, Launch instance, Preview code

- Connect to the instance and install the required things
- Update the system

```
sudo yum update -y
```

- Install Python 3 and pip (Amazon Linux)

```
sudo yum install -y python3 python3-pip
```

```
python3 -m pip install --user \
```

```
mysql-connector-python \
```

```
redis \
```

```
Faker
```

- Open Amazon RDS
- Login to AWS Console
- Search for RDS
- Open Amazon RDS
- Click Create Database
- Choose Full Configuration

The screenshot shows the 'Databases' page in the AWS Management Console. The 'Create database' button is highlighted, and a dropdown menu is open showing three options: 'Express configuration', 'Full configuration', and 'Restore from S3'.

- Choose Database Creation Method
- Select:
- Standard Create
- Select Engine
- Choose:

- Engine Type: MySQL
- Version:
- MySQL 8.0.x
- (Use the latest supported version)

Create database [Info](#)

Engine options

Engine type [Info](#)

☐ Aurora (MySQL Compatible)

☐ Aurora (PostgreSQL Compatible)

☒ MySQL

☐ PostgreSQL

☐ MariaDB

☐ Oracle

☐ Microsoft SQL Server

☐ IBM Db2

Choose a database creation method

☒ Full configuration
 You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ Easy create
 Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

- Select Template
- Choose:
- Free Tier
- (or Dev/Test if Free Tier is unavailable)

Templates

Choose a sample template to meet your use case.

☐ Production
 Use defaults for high availability and fast, consistent performance.

☐ Dev/Test
 This instance is intended for development use outside of a production environment.

☒ Free tier
 Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. [Info](#)

Availability and durability

Deployment options [Info](#)

Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the [Amazon RDS service level agreement \(SLA\)](#).

☐ Multi-AZ DB cluster deployment (3 instances)
 Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:

- 99.95% uptime
- Redundancy across Availability Zones
- Increased read capacity
- Reduced write latency

☐ Multi-AZ DB instance deployment (2 instances)
 Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:

- 99.95% uptime
- Redundancy across Availability Zones

☒ Single-AZ DB instance deployment (1 instance)
 Creates a single DB instance without standby instances. This setup provides:

- 99.5% uptime
- No data redundancy

- Configure DB Instance
- DB Instance Identifier
 - demo-mysql
- Master Username
 - admin
- Choose Self-Managed
- Master Password
- Confirm Password

Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☐ Managed in AWS Secrets Manager - *most secure*
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed
Create your own password or have RDS create a password that you manage.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / * @

Confirm master password [Info](#)

- Instance Configuration
 - Choose Burstable classes
- Choose:
 - db.t3.micro
- (or any supported instance class)

Instance configuration
The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class [Info](#)

▼ Hide filters

☐ Show instance classes that support Amazon RDS Optimized Writes [Info](#)
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

☐ Include previous generation classes

☒ Burstable classes (includes t classes)

Instance type

2 vCPUs 1 GiB RAM EBS Bandwidth: Up to 2,085 Mbps Network: Up to 5 Gbps

- Storage Configuration
- Storage Type
- General Purpose SSD (gp3)
- Allocated Storage
- 20 GB

Storage

Storage type [Info](#)
Provisioned IOPS SSD (io2) storage volumes are now available.

Baseline performance determined by volume size

Allocated storage [Info](#)

GIB

Allocated storage value must be 20 GiB to 6,144 GiB

► Additional storage configuration

- Connectivity
- In VPC, select the one you have created.
- or your custom VPC.
- Public Access
- NO

Connectivity [Info](#)

Compute resource
Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ **Don't connect to an EC2 compute resource**
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ **Connect to an EC2 compute resource**
Set up a connection to an EC2 compute resource for this database.

Virtual private cloud (VPC) [Info](#)
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

RDS-VPC-vpc (vpc-0e0462813e3d4bdb0)
4 Subnets, 2 Availability Zones

Default VPC (vpc-099792461c7d0a616)
11 Subnets, 7 Availability Zones

RDS-VPC-vpc (vpc-0e0462813e3d4bdb0)
4 Subnets, 2 Availability Zones

Create new VPC
Generates a new VPC automatically when you submit the form.

Create new DB Subnet Group

Public access [Info](#)

☐ **Yes**
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☒ **No**
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

- In VPC Security Group use existing security group.
- That we have created for RDS access

VPC security group (firewall) [Info](#)
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ **Choose existing**
Choose existing VPC security groups

☐ **Create new**
Create new VPC security group

Existing VPC security groups

Choose one or more options

Q |

☒ **RDS-SG**

☐ EC2-SG

☐ Redis-SG

☐ default

Certificate authority - optional [Info](#)
Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision.

rds-ca-rsa2048-g1 (default)
Expiry: May 26, 2061

If you don't select a certificate authority, RDS chooses one for you.

- Create Database
- After the RDS becomes available
- Connect the RDS to EC2 using RDS Endpoint
- Before that install the MySQL Client

```
sudo dnf install mariadb105 -y
```

```
mysql --version
```

- Now after installation connect endpoint to EC2

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

- Enter the password that you have given in the RDS
- Check Databases

```
SHOW DATABASES;
```

- If rvnl Does Not Exist

```
CREATE DATABASE rvnl;
```

- Then:

```
USE rvnl;
```

- Create Table

```
CREATE TABLE random (  
    id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
    name VARCHAR(255),  
    email VARCHAR(255),  
    address VARCHAR(255),  
    phone VARCHAR(255),  
    job_title VARCHAR(255),  
    date_of_birth DATE,  
    info TEXT,  
    PRIMARY KEY (id)  
);
```

- Verify:

```
SHOW TABLES;
```

```
MySQL [(none)]> CREATE DATABASE rvnl;  
Query OK, 1 row affected (0.053 sec)  
  
MySQL [(none)]> USE rvnl;  
Database changed  
MySQL [rvnl]> CREATE TABLE random (  
-> id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
-> name VARCHAR(255),  
-> email VARCHAR(255),  
-> address VARCHAR(255),  
-> phone VARCHAR(255),  
-> job_title VARCHAR(255),  
-> date_of_birth DATE,  
-> info TEXT,  
-> PRIMARY KEY (id)  
-> );  
Query OK, 0 rows affected (0.102 sec)  
  
MySQL [rvnl]> SHOW TABLES;  
+-----+  
| Tables_in_rvnl |  
+-----+  
| random          |  
+-----+  
1 row in set (0.001 sec)  
  
MySQL [rvnl]> █
```

- You need to populate the random table with around 210,000 records.
- Create the Python Script and change the endpoint and password.
- On EC2:

```
vim load_data.py
```

- Paste:

```
from faker import Faker
```

```
import mysql.connector

fake = Faker()

conn = mysql.connector.connect(
    host="YOUR-RDS-ENDPOINT",
    user="admin",
    password="YOUR_PASSWORD",
    database="rvnl"
)

cursor = conn.cursor()

batch_size = 5000
total_records = 210000
for batch_start in range(0, total_records, batch_size):
    data = []

    for _ in range(batch_size):
        data.append((
            fake.name(),
            fake.email(),
            fake.address(),
            fake.phone_number(),
            fake.job(),
            fake.date_of_birth(minimum_age=18, maximum_age=65),
            fake.text()
        ))

    cursor.executemany("""
        INSERT INTO random
```

```

        (
            name,
            email,
            address,
            phone,
            job_title,
            date_of_birth,
            info
        )
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s)

"""", data)

conn.commit()

print(f"Inserted {batch_start + batch_size:,} records")

cursor.close()

conn.close()

print("Completed")

```

- Run:

```
python3 load_data.py
```

- Now connect to the MySQL client again

```

MySQL [(none)]> USE rvnl;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [rvnl]>
MySQL [rvnl]> SELECT COUNT(*) FROM random;
+-----+
| COUNT(*) |
+-----+
| 210000 |
+-----+
1 row in set (0.050 sec)

MySQL [rvnl]> 

```

- Now click on the databased that you have created in the RDS
- Click Action

demo-mysql

  [Modify](#) [Actions](#) ▼

- Click ElastiCache cluster
- Cluster Type
 - Select: Redis OSS
- Deployment Option
 - Select: Design your own cache
- Cluster Mode
 - Cluster mode disabled

Create ElastiCache cluster: demo-mysql [info](#)

Create a compatible ElastiCache cluster that inherits settings from your source DB. You can save up to 55% in cost and gain up to 80x faster read performance using ElastiCache with RDS for MySQL (vs. RDS for MySQL alone).

ElastiCache cluster configuration [info](#)

Source DB identifier
demo-mysql

Cluster type

☐ Valkey (recommended)

☒ Redis OSS

☐ Memcached

Deployment option

☐ Serverless cache

Use to quickly create a cache that automatically scales to meet application traffic demands, with no servers to manage.

☒ Design your own cache

Use to create a cache by selecting node type, size, and count.

Cluster mode

☐ Enabled

Cluster mode enables replication across multiple shards for enhanced scalability and availability.

☒ Disabled

The Redis cluster will have a single shard (node group) with one primary node and up to 5 read replica.

 If you choose 'Cluster mode turned off', you cannot change the number of shards. The configuration supports all Redis commands and functionality but limits maximum cache size and performance. [Learn more](#) 

- Node Type
 - cache.t4g.micro
- Replicas
 - 0

Name

demo-redis

The name can have up to 40 characters, and must not contain spaces.

Description - optional

Description

Engine version

Version compatibility of the Redis engine that will run on your nodes.

7.1

Node type

The type of node to be deployed and its associated memory size.

cache.t4g.micro

0.5 GiB memory Up to 5 Gigabit network performance

Number of replicas

Enter the number of replicas between 0 and 5. Zero replicas will not enable an enhanced cluster with primary/replica roles.

0

 The selected node type, the number of shards, and the number of replicas all affect your cluster performance and resource costs. Be sure these settings match your database needs. [Learn more](#) 

- ElastiCache connectivity settings
 - Choose: Customize your connectivity settings
- Now in subnet groups
 - Choose: Create a new subnet group
- Then select the VPC we have created

ElastiCache connectivity settings [Info](#)

You can configure your connectivity settings by using inherited and default values or define your own.

☐ ElastiCache default connectivity settings
Use the following default values.

☒ Customize your connectivity settings
Edit the default values according to your requirements.

Port
The port number on which nodes accept connections.

6582

Network type
Choose between IPv4, dual stack, and IPv6.

IPv4
Your resources will only communicate over the IPv4 protocol.

Subnet groups

☐ Choose existing ElastiCache subnet group

☒ Create a new subnet group

Changing the subnet group and selecting a different VPC might affect how you connect to your ElastiCache cluster after creation.

Subnet groups

☐ Choose existing ElastiCache subnet group

☒ Create a new subnet group

Changing the subnet group and selecting a different VPC might affect how you connect to your ElastiCache cluster after creation.

Name

redis-subnet-group

The name is required, can have up to 255 characters and must not contain spaces.

Description - optional

Created from RDS create flow

VPC ID
The identifier for the VPC environment where your cluster is to run.

vpc-0e0462813e3d4bdb0

[Create VPC](#)

For Multi-AZ high availability mode, choose IDs for at least two subnets from two Availability Zones in the table below.

- ElastiCache security settings
 - Choose: Customize your security settings
- Leave the rest of the things as it is

ElastiCache security settings [Info](#)

You can configure your network and data security settings by using default ElastiCache values or define your own.

☐ ElastiCache default security settings
Use the following recommended ElastiCache values.

☒ Customize your security settings
Edit the default ElastiCache values according to your requirements.

Encryption at rest

☒ Enable
Enables encryption of data stored on disk.

Encryption key
The master key that will be used to protect the key used to encrypt data at rest for this cluster.

☒ Default key
An AWS-owned KMS key will be used for encryption.

☐ Customer managed CMK
Select a customer managed key.

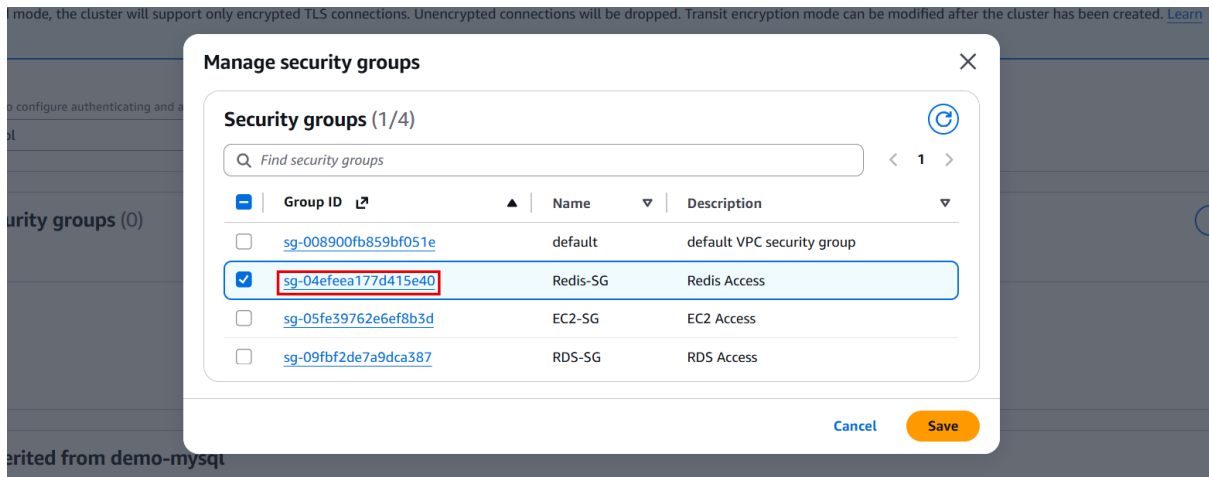
Encryption in transit

☒ Enable
Enables encryption of data that moves between the service and client.

Transit encryption mode
Required

In Required mode, the cluster will support only encrypted TLS connections. Unencrypted connections will be dropped. Transit encryption mode can be modified after the cluster has been created. [Learn more](#)

- Move to select security group
 - click Manage
- choose the security group
 - Click Save



- Click Create ElastiCache cache
- As the status becomes available
- Go to ElastiCache cache and copy the primary endpoint
- Next Step: Create mgrdstoreredis.py
- Run:

```
nano mgrdstoreredis.py
```

- Paste this code:

```
import mysql.connector
import redis
import json
from datetime import date, datetime

# RDS Connection
mysql_conn = mysql.connector.connect(
    host="demo-mysql.cghaxz7pqg8a.us-east-1.rds.amazonaws.com",
    user="admin",
    password="YOUR_RDS_PASSWORD",
    database="rvnl"
)

mysql_cursor = mysql_conn.cursor(dictionary=True)
```

```
# Redis Connection

redis_client = redis.Redis(
    host="master.demo-redis.vrl0te.usel.cache.amazonaws.com",
    port=6582,
    ssl=True,
    decode_responses=True
)

print("Connected to Redis")

mysql_cursor.execute("""
SELECT
    id,
    name,
    email,
    address,
    phone,
    job_title,
    date_of_birth,
    info
FROM random
""")

batch_size = 1000
total_loaded = 0

while True:

    rows = mysql_cursor.fetchmany(batch_size)
```

```

if not rows:
    break

pipe = redis_client.pipeline()

for row in rows:

    for k, v in row.items():
        if isinstance(v, (date, datetime)):
            row[k] = str(v)

    pipe.set(
        f"user:{row['id']}",
        json.dumps(row)
    )

pipe.execute()

total_loaded += len(rows)

print(f"Loaded {total_loaded:,} records")

mysql_cursor.close()
mysql_conn.close()

print("\nMigration Completed")
print(f"Total Records Loaded: {total_loaded:,}")

```

- Important

- Replace:
 - password="YOUR_RDS_PASSWORD"
- with your actual RDS password.
- Run

```
python3 mgrdstoredis.py
```

- Performance Comparison
- Read Directly From RDS
- Create:

```
nano readfromdatabase.py
```

```
import mysql.connector
import time

conn = mysql.connector.connect(
    host="demo-mysql.cghaxz7pqg8a.us-east-1.rds.amazonaws.com",
    user="admin",
    password="YOUR_PASSWORD",
    database="rvnl"
)

cursor = conn.cursor(dictionary=True)

job_title = "Designer, jewellery"

start = time.time()

cursor.execute("""
SELECT *
FROM random
WHERE job_title = %s
```

```
""", (job_title,))

results = cursor.fetchall()

end = time.time()

print(f'Records Found: {len(results)}')
print(f'RDS Query Time: {end-start:.6f} seconds")

cursor.close()
conn.close()
```

- Run:

```
python3 readfromdatabase.py
```

- Load those records into Redis once
- Temporarily create:

```
nano cache_job.py
```

```
import mysql.connector
import redis
import json

conn = mysql.connector.connect(
    host="demo-mysql.cghaxz7pqg8a.us-east-1.rds.amazonaws.com",
    user="admin",
    password="YOUR_PASSWORD",
    database="rvnl"
)

cursor = conn.cursor(dictionary=True)
```

```

cursor.execute("""
SELECT *
FROM random
WHERE job_title = 'Designer, jewellery'
""")

results = cursor.fetchall()

r = redis.Redis(
    host="master.demo-redis.v10te.us-east-1.cache.amazonaws.com",
    port=6582,
    ssl=True,
    decode_responses=True
)

r.set(
    "job:designer_jewellery",
    json.dumps(results, default=str)
)

print(f"Cached {len(results)} records")

```

- Run:

```
python3 cache_job.py
```

- Expected:
 - Cached 379 records
- Compare with Redis
- create file

```
nano readfromredis.py
```

```
import redis
import json
import time

r = redis.Redis(
    host="master.demo-redis.vrl0te.usel.cache.amazonaws.com",
    port=6582,
    ssl=True,
    decode_responses=True
)

start = time.time()

data = r.get("job:designer_jewellery")

records = json.loads(data)

end = time.time()

print(f"Records Found: {len(records)}")
print(f"Redis Query Time: {end-start:.6f} seconds")
```

- Run:

```
python3 readfromredis.py
```

We tested the same query against both the SQL database and Redis cache. The SQL database completed the query in approximately **0.05 seconds**, while Redis returned the result in just **0.01 seconds**.

This represents a **0.04-second reduction in response time**, making Redis about **80% faster** than the SQL database for this query.